

SEMT20001: Principles of Computational Modelling

COURSEWORK ASSESSMENT: QUESTION 3

The question below forms one part of the assessment for the unit.

This is question 3 of 4.

Each question carries 25 marks. Answer all questions.

The maximum mark for this assessment is 100 marks.

The deadline for submission for all questions is **1pm on 19 March 2026 (Week 21)**.

You must submit **two** items (on Blackboard) for **each** question:

- (1) a single self-contained Jupyter Notebook, that reports your answers to the question, and explains how you found them, including all relevant Python commands and functions, and explanations of your code design, **and**
- (2) a single .zip file containing all Python .py files.

Your Jupyter Notebook (1) for this question must contain everything you wish to be marked for this question (including, but not limited to, numerical and textual answers, figures, and all Python code used to generate them). Material not contained in your document (1) will receive no credit.

Any Python .py files that are not contained in your .zip file (2) will receive no credit.

Your submission must run from the pocm2526 environment, as defined on Blackboard (see *Learning and Teaching Area : Read me first! Python starter guide* for details). Any code that does not will receive no credit.

Your submission must be your own individual effort; submissions will be checked for possible plagiarism. You may only use tools such as spelling and grammar checkers in this assignment, and their use should be limited to corrections of your own work rather than substantial re-writes or extended contributions. Use of AI applications (even if they are free) beyond this is contract cheating; see <https://www.bristol.ac.uk/students/support/academic-advice/academic-integrity/contract-cheating/>. This is an AI Category 2 assessment; see <https://www.bristol.ac.uk/bilt/sharing-practice/guides/guidance-on-ai/using-ai-in-assessment/> for full details.

Question 3: Agent-based Modelling — Campaign for a SEMT cat!

Suppose we were to hold a vote on whether SEMT should have its very own school cat. We may have some people who are totally against or passionately in favour of a school cat, and people with opinions somewhere in between, but how can these opinions be changed and what determines the final outcome?

In this coursework question, we will be using a simple ABM representing an opinion dynamics model to see how opinions on whether or not to get a SEMT cat change as a result of social interactions.

We make the following assumptions:

- (A1) Each person a_i is characterised by a single opinion $x_i \in [-1, 1]$, signifying strongly against a school cat ($x_i = -1$) to passionately in favour of a school cat ($x_i = 1$).
- (A2) Initial opinions are uniformly distributed between -1 and 1 .
- (A3) At each timestep, every person picks one other person at random to share opinions with.
- (A4) When chatting people can have some *positive influence* on each others' opinions if they are similar enough. People update their opinions using a bounded confidence model¹.

Two people with opinions x_1 and x_2 update their opinions as follows:

$$x_1 \leftarrow \begin{cases} x_1 + \gamma(x_2 - x_1) & \text{if } |x_1 - x_2| \leq d, \\ x_1 & \text{otherwise.} \end{cases}$$
$$x_2 \leftarrow \begin{cases} x_2 + \gamma(x_1 - x_2) & \text{if } |x_1 - x_2| \leq d, \\ x_2 & \text{otherwise.} \end{cases}$$

Where γ is a learning rate, denoting the magnitude of social influence agents have upon each other, and d is the confidence threshold, the minimum difference in opinions to warrant social influence.

You have been given the following three Python files:

- **OpinionModel.py** : This contains the **OpinionWorld** model class. The **OpinionWorld** class has two methods: `__init__` and `step`. Your answers should only require you to edit the `__init__` method, or write new methods.
- **OpinionAgent.py** : This contains the **Agent** class. The **Agent** class has three methods: `__init__`, `update_opinion` and `chat`. Your answers should only require you to edit the `__init__` and `update_opinion` methods, or write new methods.
- **OpinionApp.py** : This contains the code used to create the simulation visualisation. You can run it using `solara run OpinionApp.py`. Your answers should not require you to edit this file.

All written answers and any code used to produce results and figures should be submitted in a Jupyter Notebook. All additions/changes to the given Python files should be clearly commented.

In all your answers, you should ensure that your results are reproducible, by giving all relevant methods, model parameters, random number seeds², etc. Unless explicitly stated, all model parameters should be kept as the default values. Also, don't forget to think carefully about how long to run the simulations and the number of replications needed when answering each question.

¹Inspired by Deffuant, G., Neau, D., Amblard, F. and Weisbuch, G., 2000. Mixing beliefs among interacting agents. *Advances in Complex Systems*, 3(01n04), pp.87-98.

²Use `rng = np.random.default_rng(42)`.

- (a) Run the given model, with no modifications or changes to the default parameters, for a single run and produce a line plot showing how the agent opinions change over time. Does the population opinion converge as expected? Discuss.

(2 marks)

- (b) The default model sets `confidence_threshold=2.0` meaning that people can be influenced by anyone, regardless of their opinions. However, in the bounded confidence model, agents who are sufficiently dissimilar will simply ignore each other.

We can explore this effect by varying `confidence_threshold` d .

- (i) Does the population still always reach consensus? How does the behaviour change as `confidence_threshold` changes? Demonstrate this by producing line plots showing how agent opinions change over time (as for part (a)) for two *different* values of `confidence_threshold`.

(3 marks)

- (ii) Rather than converging to a single consensus view, populations can often divide into several distinct cliques of like-minded members, but when does this happen?

Produce a plot showing the number of cliques that emerge under the bounded confidence model as a function of `confidence_threshold`, where $d \in [0.1, 0.9]$.

Use the following function to calculate the number of cliques in a population:

```
def ret_num_cliques(opinions, confidence_threshold):
    # begin with zero cliques
    num_cliques = 0

    # sort opinions in increasing order, ignoring outliers
    sorted_constrained_opinions = sorted([opinion for opinion in opinions if abs(↵
        opinion) <= (1 - (0.5 * confidence_threshold))])

    # set the minimum opinion value
    x_min = sorted_constrained_opinions[0]

    for opinion in sorted_constrained_opinions:
        # check if there are any opinions outside of the current range of influence, ↵
        # i.e. different clique
        if opinion > (x_min + confidence_threshold):
            # if yes, update minimum opinion value and number of cliques
            x_min = opinion
            num_cliques += 1

    return num_cliques
```

(3 marks)

- (c) Assumption (A3) states a person chooses a partner to share opinions with at random, but are social interactions really random across the entire population? Often a person's social interactions are limited to partners within a certain "neighbourhood".

- (i) Fix `confidence_threshold= 0.3` and investigate the effect of varying the model parameter `neighbourhood_size` by producing:

- Line plots from a single run showing how the agent opinions change over time (as in part (a)) for different values of `neighbourhood_size`.
- Boxplots across multiple runs showing the relationship between the final proportion of extremists³ and `neighbourhood_size`.

Comment on your findings, including any qualitative observations you might make using the visualisation.

(4 marks)

- (ii) Describe the design choices that were made when implementing “space” in the default model. Discuss how you would implement this using networks instead. (You do not need to actually implement.)

(2 marks)

- (d) Assumption (A2) states that the initial distribution of peoples’ opinions is uniformly distributed between -1 and 1 .

- How reasonable is this assumption? Discuss.

(2 marks)

- Now implement an initial distribution of your choosing in the model, using the model attribute `initialisation_type` to switch between initial distributions. Describe and explain the distribution (including any parameters), why you have chosen it, and how you have verified that your implementation works correctly.

Run the model with your new initial distribution for a single run, keeping all other parameters to the default, and produce a line plot showing how the agent opinions change over time. Compare and contrast with part (a) - does the change in behaviour match your expectations?

(4 marks)

- (e) Sadly, despite my impassioned campaigning, the final vote resulted in a negative final consensus and so we will not be getting a SEMT cat. From my observations, I believe this was due to a proportion of stubborn negative voters who refused to change their opinion, no matter who they interacted with.

Implement this ‘stubborn’ behaviour in the default model to test my hypothesis and explore what proportion of stubborn negative voters results in the observed behaviour. Use the model attribute `initialisation_type` to switch between stubborn and default initialisations and add any necessary model or agent attributes. Describe and explain your design choices, implementation, and how you have verified that it works correctly.

(5 marks)

³We define an extremist as an agent with opinion $|x| > 0.9$.